



Welcome to CSC 365

Internet Programming

Dr. Yifan Zhang

React Props Children

In React, `props.children` is a special prop that allows components to receive and render content passed between their opening and closing tags.

Children Example

```
16 function Person(props) {
17   const box = { border: 'dotted', padding: '1rem', ...(props.style || {}) };
18   return (
19     <div style={box}>
20       <h2>{props.name}</h2>
21       <p>{props.age} years old</p>
22       <p>Works as a(n) {props.job}</p>
23       <div>{props.children}</div>
24     </div>
25   );
26 }
27
28 export default Person
```

```
22 function Random_Person() {
23   return <Person name="Random Rick" age={42} job="Engineer" />;
24 }
25
26 export default Random_Person
```

Children Example

In React, you can send the content between the opening and closing tags of a component, to another component.

```
src > Props_Children.jsx > ParentComponent
1  import Person from './App.jsx'
2  import Random_Person from './Person.jsx'
3
4
5  function ParentComponent() {
6    return (
7      <>
8        <div>
9          <h1>This text is the first child content.</h1>
10         <Person name="Yifan Zhang" age={32} job="Instructor">
11           <p>
12             This was written in the Parent component, but displayed as a part of
13             the Person component
14           </p>
15         </Person>
16       </div>
17
18       <div style={{ border: '1px solid black', padding: '10px' }}>
19         <Random_Person />
20       </div>
21     </>
22   );
23 }
24
25 export default ParentComponent
```

Yifan Zhang

32 years old

Works as a(n) Instructor

This was written in the Parent component, but displayed as a part of the Person component

Random Rick

42 years old

Works as a(n) Engineer

React Lists

Render lists with `Array.map()`.

```
16 function BasicList() {
17   const fruits = ["Apple", "Banana", "Cherry", "Dragonfruit"];
18
19   return (
20     <section>
21       <h2>Basic list (strings)</h2>
22       <ul>
23         {fruits.map((fruit) => (
24           // Fruit name is unique here, so it's okay as a key for this *static* list.
25           <li key={fruit}>{fruit}</li>
26         ))}
27       </ul>
28     </section>
29   );
30 }
```

React Lists

Render lists with `Array.map()`.

Basic list (strings)

- Apple
- Banana
- Cherry
- Dragonfruit

React forms

Controlled inputs (most common)

- The input's value lives in state (useState).
- You show that value with `value={...}` and update it in `onChange`.
- Result: React is the single source of truth. This makes things like live validation, enabling/disabling buttons, and showing previews easy.

React forms

What happens when the user types (controlled)

- User types a letter.
- The input's `onChange` runs and gives you the new text.
- You call the state mutator (e.g., `setName(newText)`).
- React re-renders, and the input shows the updated value.
- That loop keeps the UI and your data perfectly synced.

React forms

Controlled inputs (most common)

The input's value lives in state

You show that value with `value` prop
update it in `onChange`.

Result: React is the single source of truth

This makes things like live validation,
enabling/disabling buttons, and
previews easy.

```
src > forms_example.jsx > ...
1   import { useState } from "react";
2
3   export default function NameForm() {
4     const [name, setName] = useState("");
5
6     return (
7       <form>
8         <label>
9           Your name
10          <input
11            type="text"
12            value={name}
13            onChange={(e) => setName(e.target.value)}
14            placeholder="Ada Lovelace"
15          />
16        </label>
17
18        <p>Preview: {name || "-"}</p>
19      </form>
20    );
21  }
22
```

React Submit Forms

Forms try to reload the page by default.

In React you usually prevent that with `e.preventDefault()` and handle the data yourself (send it to a server, show a message, clear the form, etc.).

React Submit Forms

```
src >  forms_example.jsx >  NameForm
1   import { useState } from "react";
2
3   export default function NameForm() {
4     const [form, setForm] = useState({ name: "" });
5
6     function handleChange(e) {
7       setForm({ ...form, name: e.target.value });
8     }
9
10    function handleSubmit(e) {
11      e.preventDefault();
12      alert(`Submitted: ${form.name || "(empty)"}`);
13    }
14
15    return (
16      <form onSubmit={handleSubmit}>
17        <div>
18          <label htmlFor="name">Your name</label>
19          <input
```

React Submit Forms

Your name

Preview: —

Save

localhost:5173 says

Submitted: Yifan Zhang

Your name

Preview: **Yifan Zhang**

Save

OK

React Select

What a `<select>` looks like in React:

- Use it like any other controlled input: 1) Keep the selected value in state. 2) Bind it with `value={...}`. 3) Update it in `onChange`.
- The event gives you the selected string at `e.target.value`. Convert if you need a number.

React Select

```
src >  select_example.jsx > ...
1   import { useState } from "react";
2
3   export default function FoodSelect() {
4     const [food, setFood] = useState("");
5     return (
6       <form onSubmit={(e) => { e.preventDefault(); alert(food || "(empty)"); }}>
7         <label>
8           Favorite fast food
9           <select value={food} onChange={(e) => setFood(e.target.value)}>
10             <option value="">-- choose --</option>
11             <option value="pizza">Pizza</option>
12             <option value="burger">Burger</option>
13             <option value="taco">Taco</option>
14           </select>
15         </label>
16         <button type="submit">Save</button>
17       </form>
18     );
19   }
```

React Select

Favorite fast food

✓ — choose —

Pizza

Burger

Taco

Save

localhost:5173 says

(empty)

OK

Javascript fetch

`fetch(url, options?)` is a built-in, promise-based API for making HTTP requests (GET/POST/etc.). It returns a Response object; you then read the body with `.json()`, `.text()`, `.blob()`, etc.

```
async function getTodos() {  
  const res = await fetch('https://jsonplaceholder.typicode.com/todos?_limit=3');  
  if (!res.ok) throw new Error(`HTTP ${res.status}`);  
  const data = await res.json(); // parse body  
  console.log(data);  
}  
getTodos().catch(console.error);
```

▼ (3) [{...}, {...}, {...}] ⓘ

- ▶ 0: {userId: 1, id: 1, title: 'delectus aut autem', completed: false}
- ▶ 1: {userId: 1, id: 2, title: 'quis ut nam facilis et officia qui', completed: false}
- ▶ 2: {userId: 1, id: 3, title: 'fugiat veniam minus', completed: false}
- length: 3
- ▶ [[Prototype]]: Array(0)

useEffect in the Component Lifecycle

Used to run a function after commitment, specifically to synchronize with external systems, e.g. fetch 'ing data from an API.

```
useEffect(cb, deps)
```

cb (callback): The callback function we wish to run.

deps (dependency array): tells React when to run the effect. React compares each value in this array between renders

How deps changes the behavior

Runs after every render

```
useEffect(() => {  
  console.log('Ran after every render');  
}); // no deps array
```

Runs once on mount (and cleanup on unmount)

```
useEffect(() => {  
  console.log('Mounted once');  
  return () => console.log('Unmounted'); // cleanup  
}, []); // empty deps
```

Runs when specific values change

```
useEffect(() => {  
  console.log('name changed:', name);  
}, [name]); // only when `name` changes
```

useEffect Anti-Patterns

Use useEffect to synchronize with an external system, not to represent derived state!

This is an anti-pattern!

```
function Person() {  
  const [name, setName] = useState("James");  
  const [derivedInitial, setDerivedInitial] = useState("J");  
  
  useEffect(() => {  
    setDerivedInitial(name[0]);    // ❌ derives state from state via effect  
  }, [name]);  
  
  // ...  
}
```

useEffect Anti-Patterns

Problems:

Redundant state: `derivedInitial` is always `name[0]`; you can compute it directly.

Extra renders: Changing `name` triggers an effect that sets `derivedInitial`, which triggers a second render.

Bug-prone: Easy to get out of sync or cause loops if dependencies are wrong.

```
function Person() {  
  const [name, setName] = useState("James");  
  const [derivedInitial, setDerivedInitial] = useState("J");  
  
  useEffect(() => {  
    setDerivedInitial(name[0]); // ❌ derives state from state via effect  
  }, [name]);  
  
  // ...  
}
```

useEffect Anti-Patterns

This is preferred.

```
function Person() {  
  const [name, setName] = useState("James");  
  const initial = name[0];           // ✅ derive directly  
  
  return (  
    <div>  
      <p>Name: {name}</p>  
      <p>Initial: {initial}</p>  
    </div>  
  );  
}
```

useEffect Anti-Patterns Why avoid them?

- Harder to read/maintain
- Slower (two real DOM commits)
- The React docs recommend against it

What to do instead

If a value can be computed from existing props/state, compute it during render (or memoize with `useMemo` if it's expensive). Don't store it in state and don't compute it in an effect.

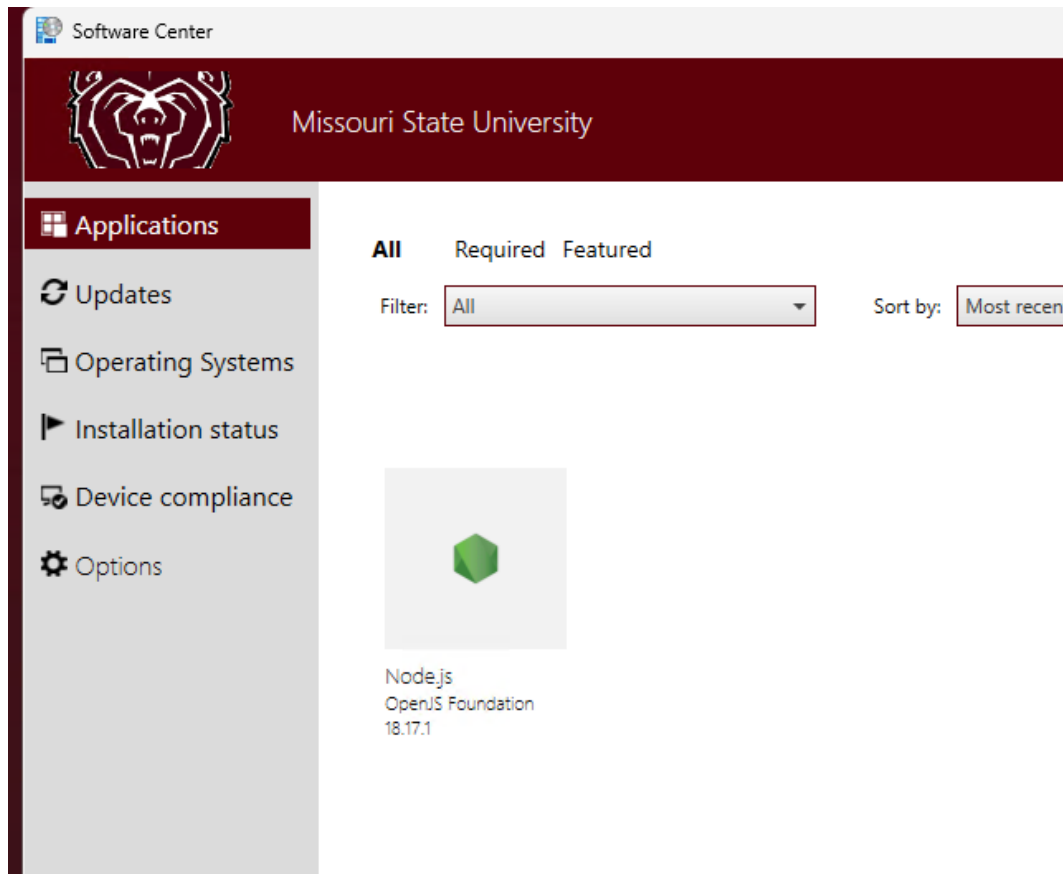
useEffect Anti-Patterns Why avoid them?

```
function Person({ name }) {  
  const [initial, setInitial] = useState('');  
  useEffect(() => setInitial(name[0]), [name]); // ❌ causes second commit  
  return <p>Initial: {initial}</p>;  
}
```

```
function Person({ name }) {  
  const initial = name[0]; // ✅ derive directly  
  return <p>Initial: {initial}</p>;  
}
```

Download Node.js on Lab Desktops

It's available to install from Software Center in the labs.



ICA 9